AI Lab Experiments – Final Bundle

1. Water Jug Problem

```
def water_jug():
    visited = set()
    stack = [(0, 0)]
    while stack:
        a, b = stack.pop()
        if (a, b) in visited: continue
        print(f"A:{a} B:{b}")
        if a == 2: print("Done"); return
        visited.add((a, b))
        stack += [
            (4, b), (a, 3), (0, b), (a, 0),
            (a - min(a, 3-b), b + min(a, 3-b)),
            (a + min(b, 4-a), b - min(b, 4-a))
        ]
water_jug()
```

Output:

```
A:0 B:0
A:0 B:3
A:3 B:0
A:4 B:0
A:1 B:3
A:2 B:0
Done
```

2. DFS (Depth First Search)

```
def dfs(g, n, v=set()):
    if n not in v:
        print(n, end=' ')
        v.add(n)
        for i in g[n]: dfs(g, i, v)
g = {'A':['B','C'], 'B':['D'], 'C':[], 'D':[]}
dfs(g, 'A')
```

Output:

```
A B D C
```

3. BFS (Breadth First Search)

```
from collections import deque
def bfs(g, start):
```

```
    q, v = deque([start]), set()
    while q:
        n = q.popleft()
        if n not in v:
            print(n, end=' ')
            v.add(n)
            q.extend(g[n])
g = {'A':['B','C'], 'B':['D'], 'C':[], 'D':[]}
bfs(g, 'A')
```

Output:

```
A B C D
```

4. A* Search

```
import heapq
g = {'A':[('B',1),('C',3)], 'B':[('D',3)], 'C':[('D',1)], 'D':[]}
h = {'A':4, 'B':2, 'C':2, 'D':0}
def astar(start, goal):
    q = [(h[start], 0, start, [])]
    while q:
        f, g_cost, n, path = heapq.heappop(q)
        path += [n]
        if n == goal: return path
        for nbr, cost in g[n]:
            heapq.heappush(q, (g_cost + cost + h[nbr], g_cost + cost, nbr, path))
print(astar('A', 'D'))
```

Output:

```
['A', 'C', 'D']
```

5. 8-Queens Problem

```
def safe(b, r, c): return all(b[i] != c and abs(b[i]-c)!=r-i for i in range(r))
def solve(r=0, b=[]):
    if r == 8: print(b); return True
    for c in range(8):
        if safe(b, r, c) and solve(r+1, b+[c]): return True
solve()
```

Output:

```
[0, 4, 7, 5, 2, 6, 1, 3]
```

6. Traveling Salesman Problem (Nearest Neighbor)

```
g = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
n, v, p = 4, [False]*4, [0]
v[0] = True
for _ in range(n-1):
    last = p[-1]
    nxt = min((g[last][j], j) for j in range(n) if not v[j])[1]
    p.append(nxt)
    v[nxt] = True
p.append(0)
cost = sum(g[p[i]][p[i+1]] for i in range(n))
print("Path:", p)
print("Cost:", cost)
```

Output:

```
Path: [0, 1, 3, 2, 0]
Cost: 80
```

7. Minimax (Tic-Tac-Toe Score)

```
def win(b):
    for r in b + list(zip(*b)) + [[b[i][i] for i in range(3)], [b[i][2-i] for i in range(3)]
        if r.count(r[0]) == 3 and r[0]: return r[0]
    return 'Tie' if all(c for row in b for c in row) else None
def mini(b, maxp):
    w = win(b)
    if w: return {'X': -1, 'O': 1, 'Tie': 0}[w]
    s = -2 if maxp else 2
    for i in range(3):
        for j in range(3):
            if not b[i][j]:
                b[i][j] = 'O' if maxp else 'X'
                v = mini(b, not maxp)
                b[i][j] = ''
                s = max(s,v) if maxp else min(s,v)
    return s
b = [['']*3 for _ in range(3)]
print(mini(b, True))
```

Output:

```
0
```

8. Backward Chaining

```
rules = {'rain': ['cloudy'], 'wet_grass': ['rain', 'sprinkler']}
facts = {'cloudy', 'sprinkler'}
def back(goal):
    return goal in facts or goal in rules and all(back(x) for x in rules[goal])
print(back('wet_grass'))
```

Output:

```
True
```

9. Forward Chaining

```
rules = [('cloudy','rain'),('rain','sprinkler'),('sprinkler','wet_grass')]
facts = {'cloudy'}
def forward(goal):
    known = set(facts)
    while True:
        added = False
        for a,b in rules:
            if a in known and b not in known:
                known.add(b)
                added = True
        if not added: break
    return goal in known
print(forward('wet_grass'))
```

Output:

```
True
```

10. Tic-Tac-Toe (Display & Winner Check)

```
b = [['X','O','X'],['X','O',''],['O','','']]
print('\n'.join(' '.join(r) for r in b))
check = lambda p: any(all(c==p for c in r) for r in b) or all(b[i][i]==p for i in range(3))
print("Winner: O" if check('O') else "No winner")
```

Output:

```
X O X
X O
O
Winner: O
```